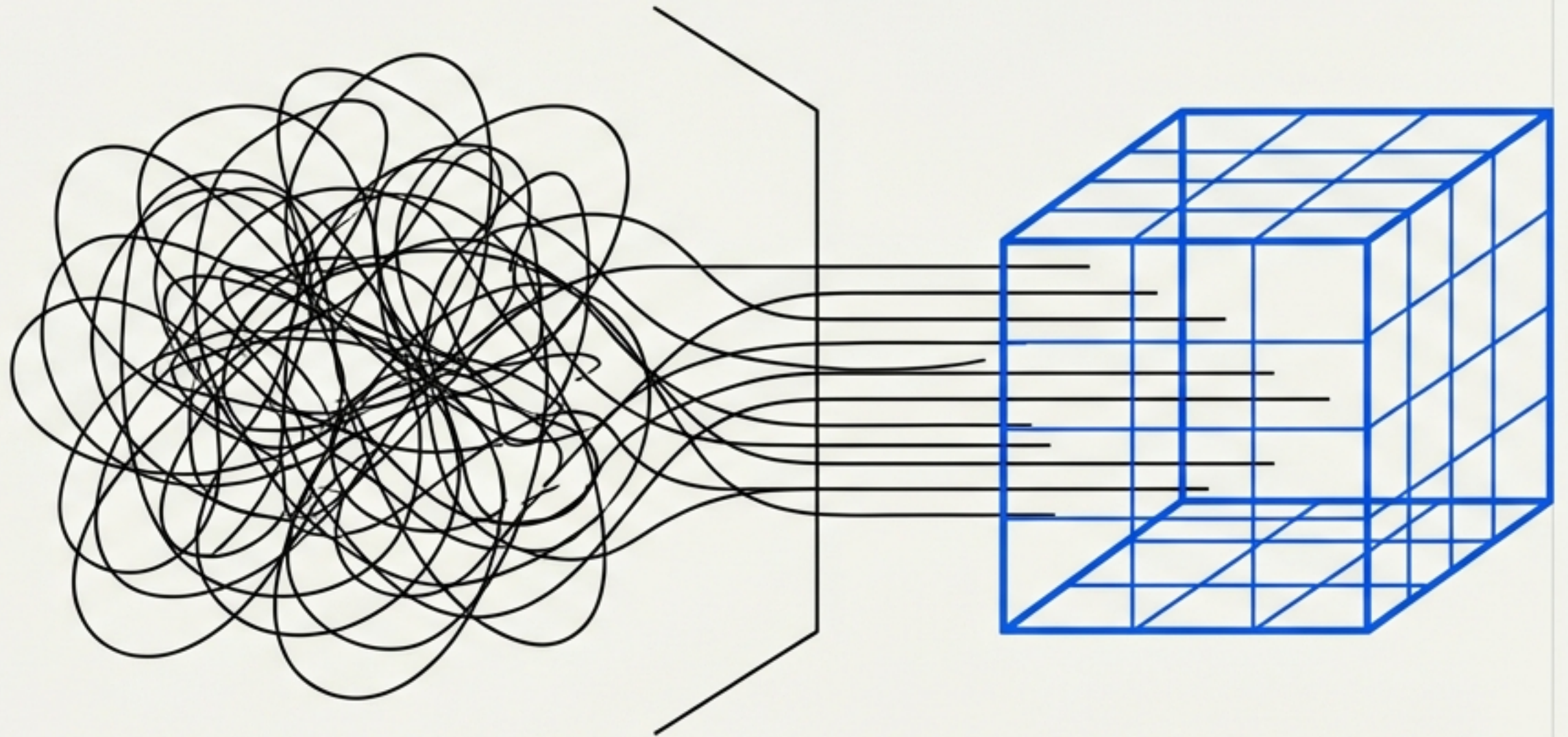


Agentic Coding Increases Your Agency Over Code

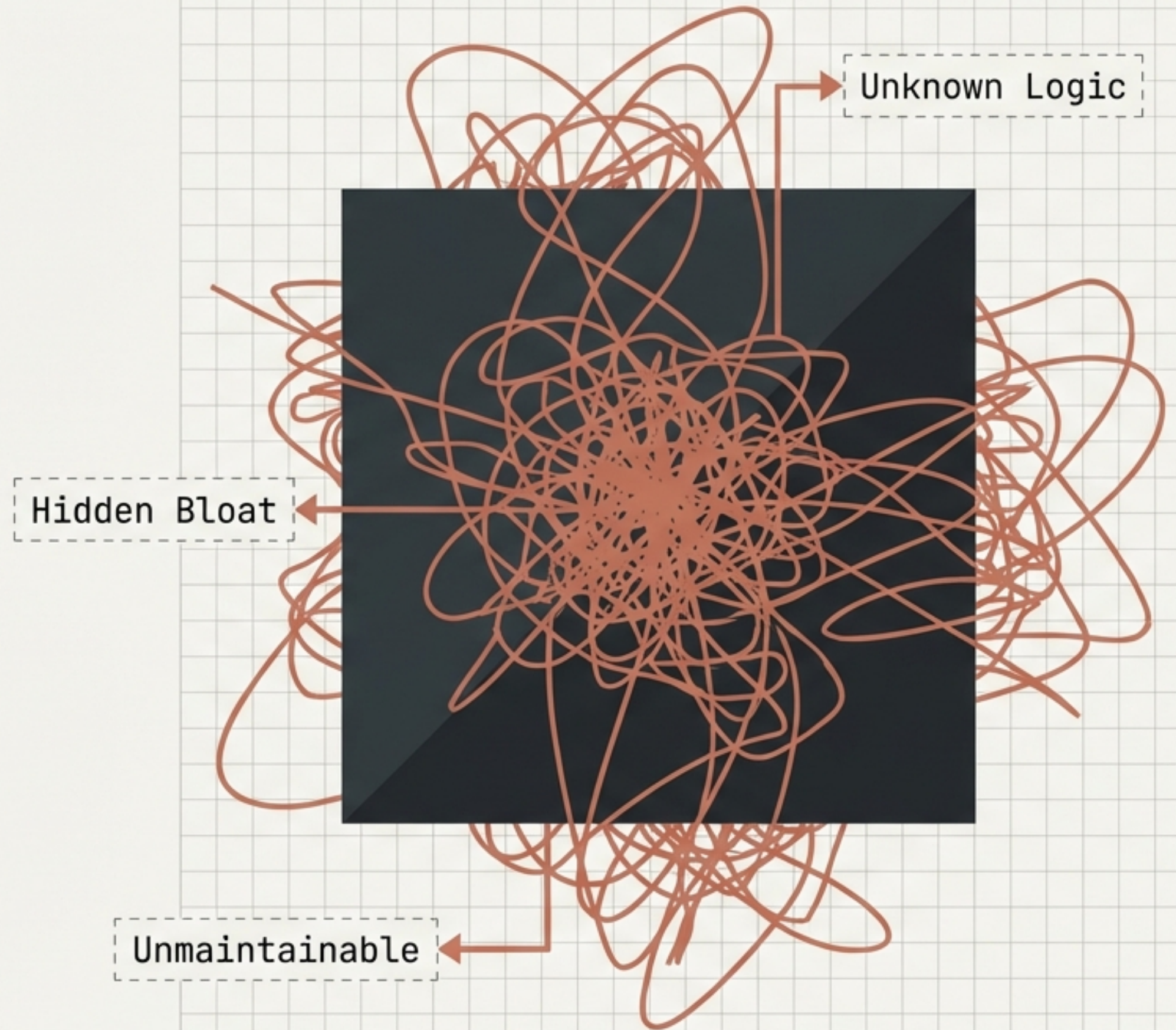
Why writing **more** code with AI actually reduces complexity and increases control.



The Prevailing Fear: A Loss of Control

The common narrative suggests that Generative AI and LLMs produce code nobody understands. The perceived wisdom is that this leads to unmaintainable bloat, a 'black box' codebase, and a total loss of control over your software.

**They say: If you didn't write it
line-by-line, you don't own it.**



The Counterintuitive Reality

In the hands of a skilled engineer, agentic coding does the opposite of what critics claim.

It moves you from “Consumer of Code” to “Owner of Code”.

Agentic coding—when properly engineered—

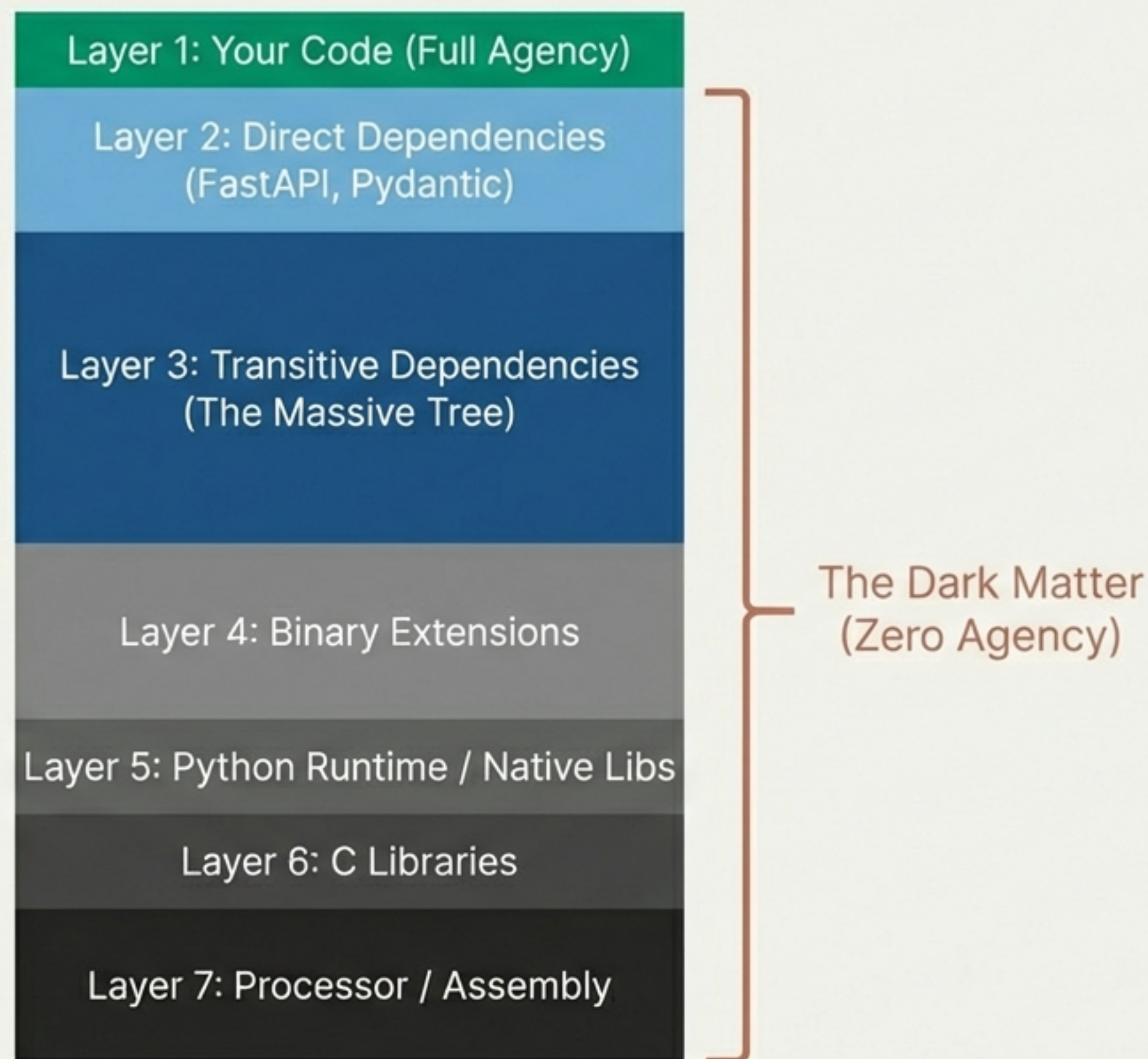
INCREASES

the amount of code you understand and control.

“This goes against a lot of the perceived wisdom... but let me show you why.”

The Honest Picture of Your Current Stack

Even before AI, the amount of code you actually understood was tiny. The majority is “dark matter” you never see.

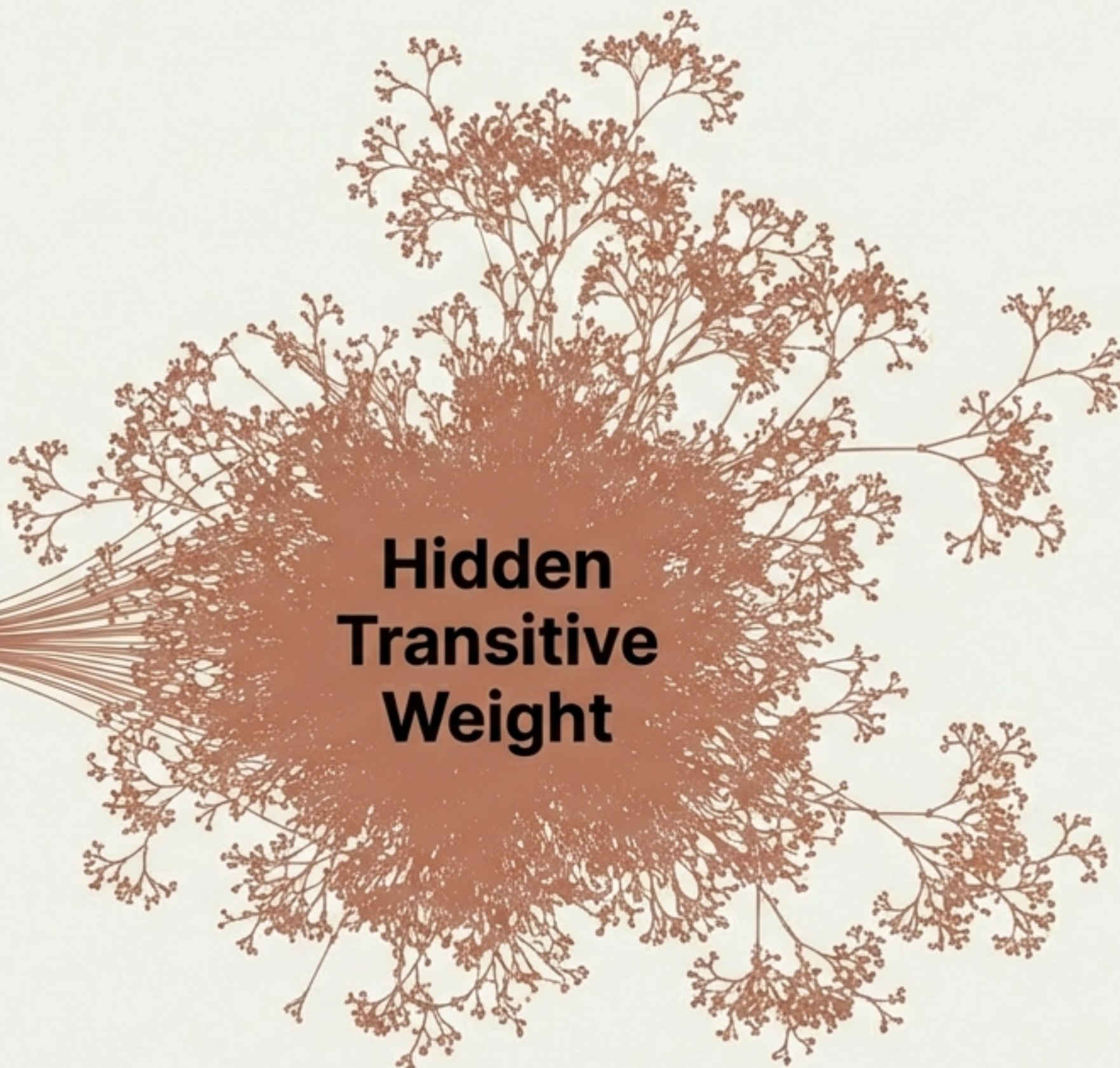


The High Cost of 'Don't Repeat Yourself'

Historically, we prioritised importing libraries over writing logic to save time. We traded effort for opacity.

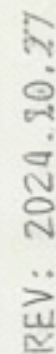
```
import simple_utility
```

You avoided writing boilerplate. Trade-off. In exchange, you accepted a massive dependency tree with zero agency.



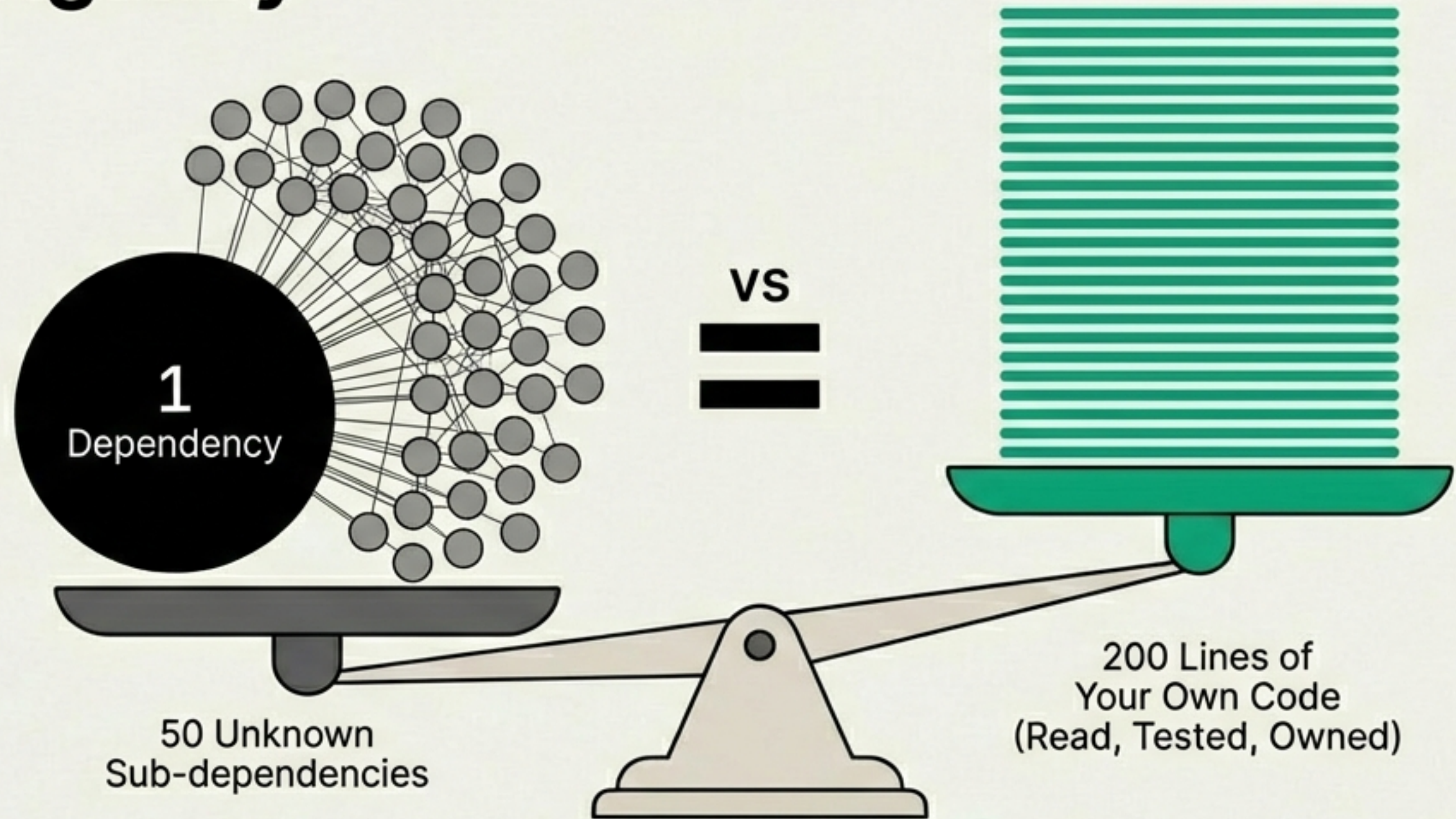
**Hidden
Transitive
Weight**

LLMs change the economics of software development. It is now feasible to implement specific functionality yourself rather than importing heavy frameworks.



The Maths of Agency

"I wrote more code"
equals "I removed a
massive black box."
This is a net gain in
system understanding.



Scenario B represents dramatically higher agency over the running system.

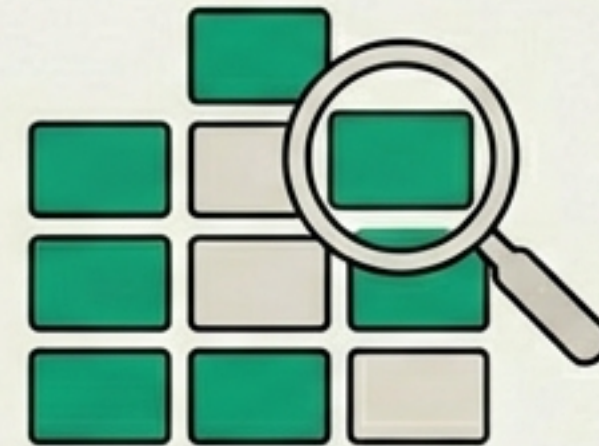
The Fork in the Road: Vibe Coding vs. Engineering

Vibe Coding (The Trap)



- Moving code into repo without understanding it.
- Trading opacity for generated slop.
- Result: Chaos.

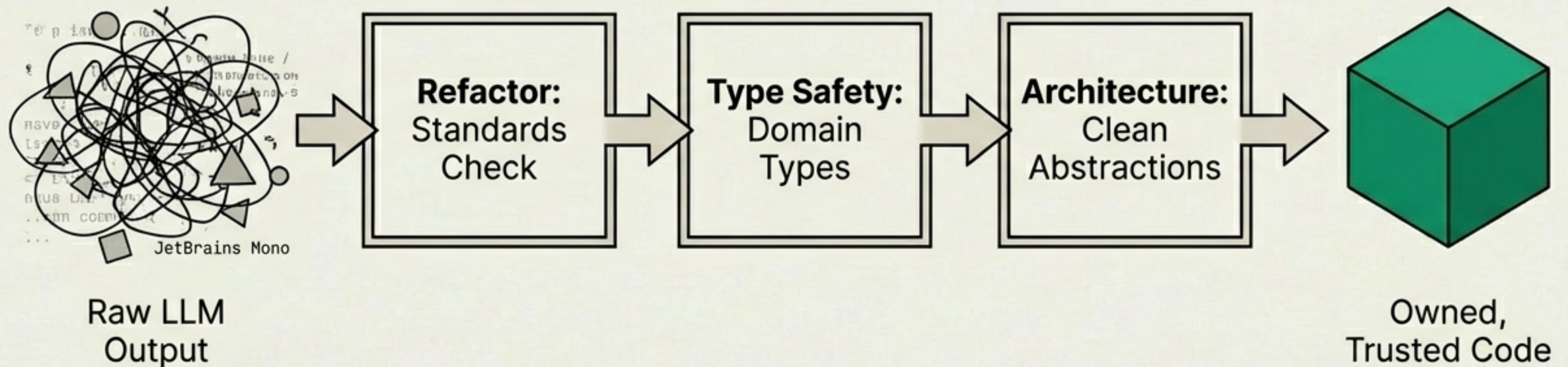
The Good Engineer (The Solution)



- Refactoring generated code.
- Enforcing visibility and understanding.
- Trading dependency opacity for **owned** transparency.

The Quality Gate

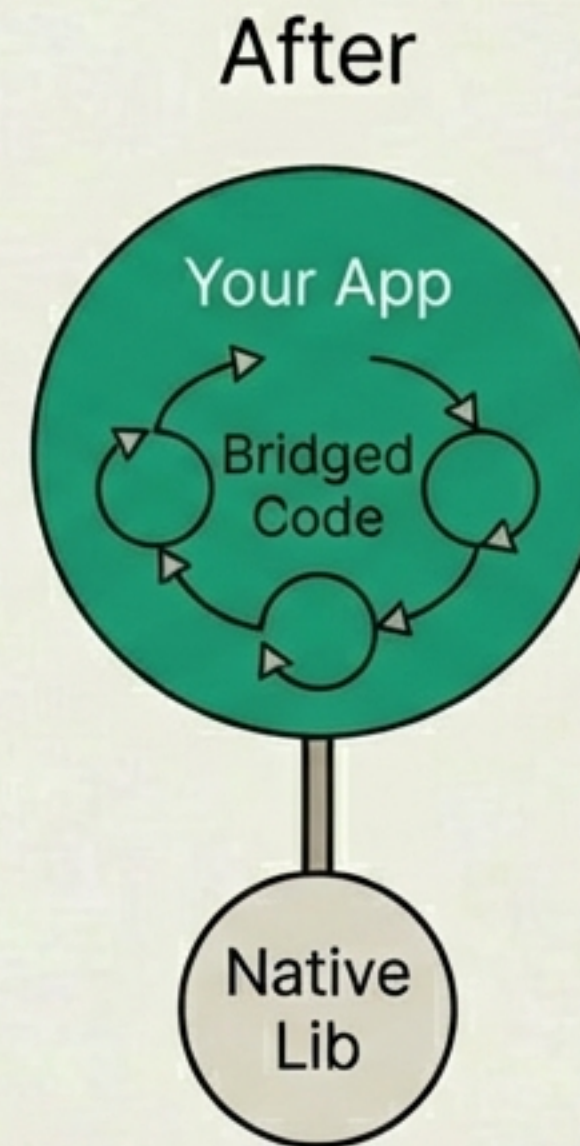
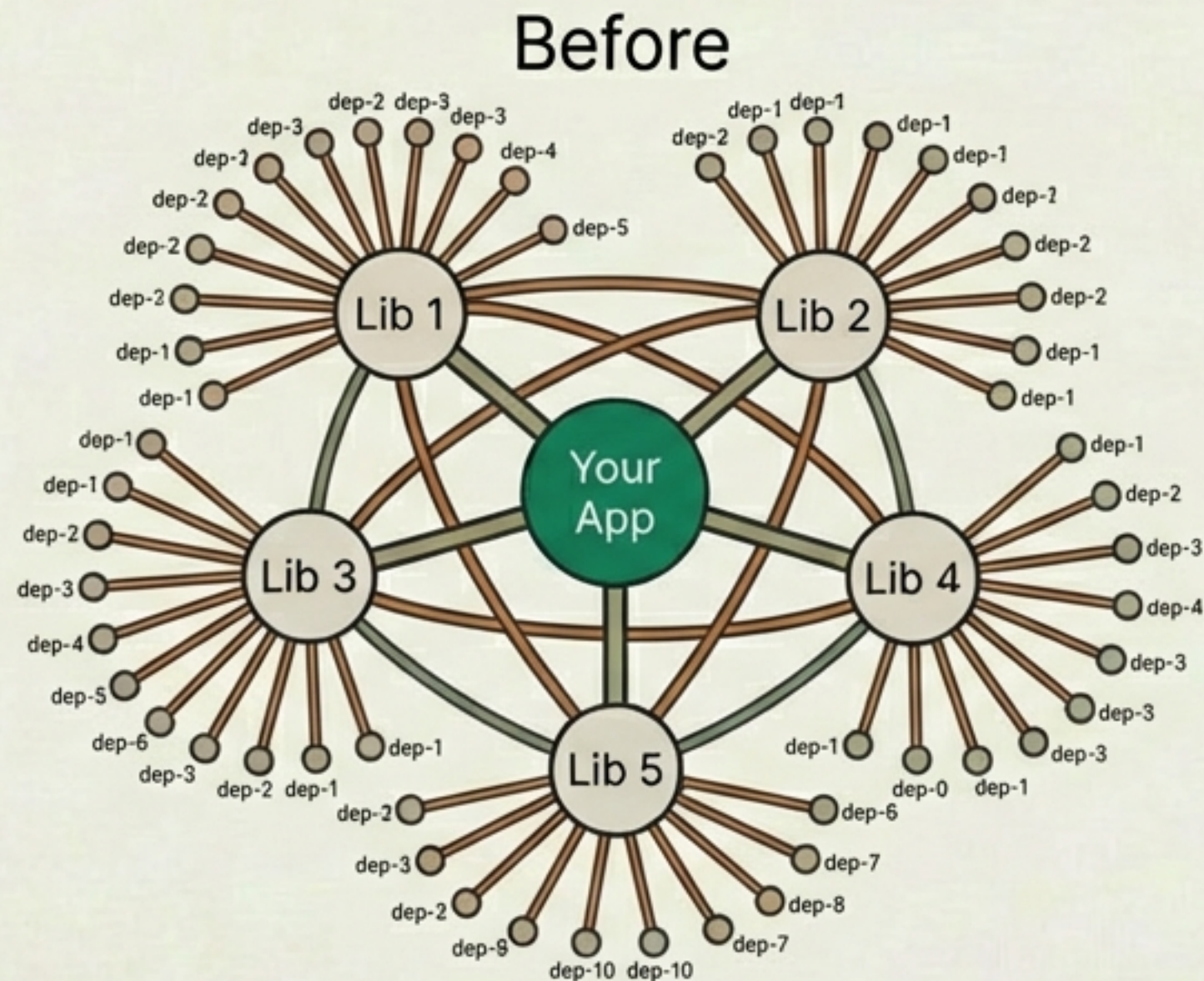
Generated code is not finished code. The LLM handles the boilerplate; you handle the architecture.



Result: More code you control. Fewer dependencies you don't.

The Dependency Reduction Pattern

The goal is to minimise the code you don't* have agency over.

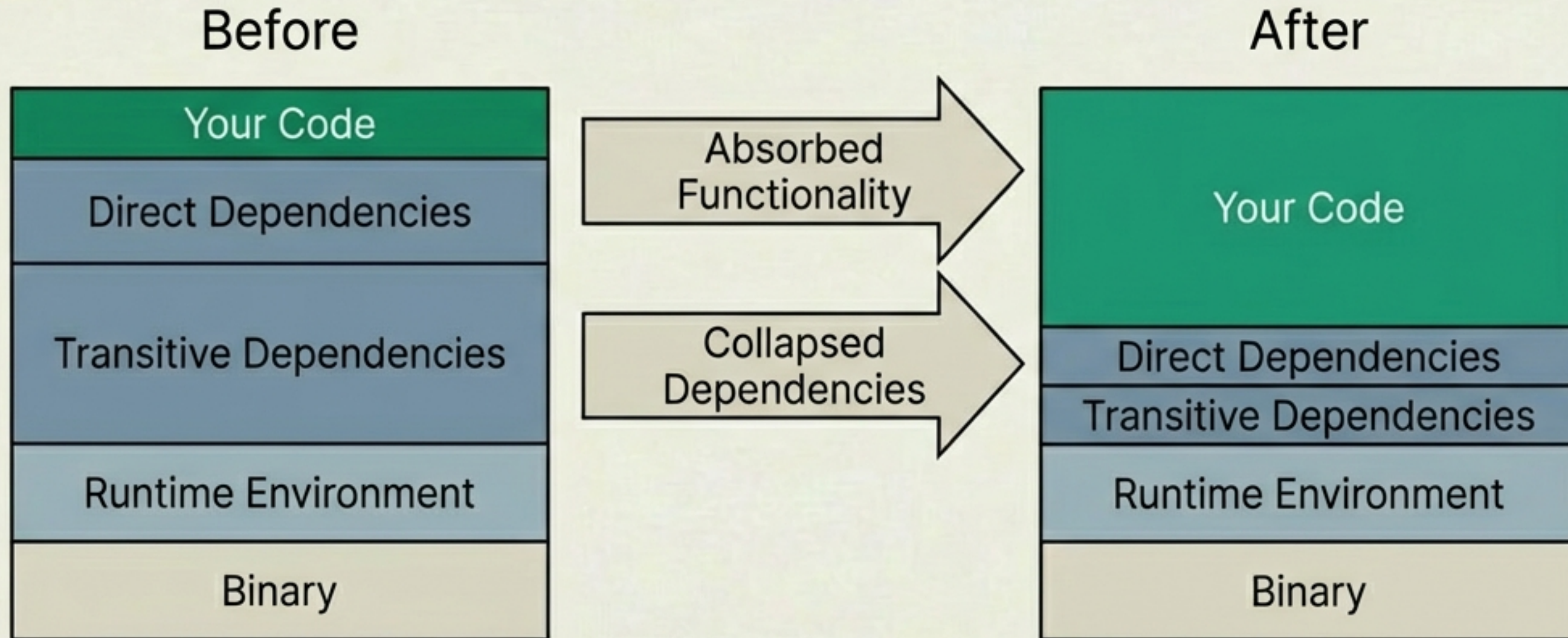


Implementation

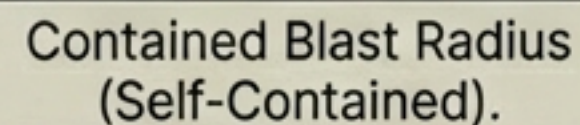
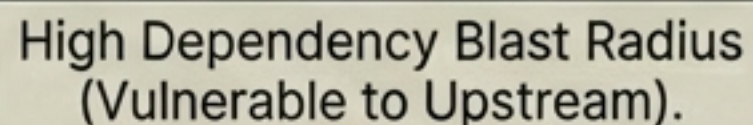
- **Python:** Stick to native libraries.
- **JavaScript:** Pure JS + HTML.
- **Bridge:** Use LLM to write the specific implementation.

The Revised Agency Stack

The total code running hasn't changed much. But the proportion you understand and control has shifted significantly upward.



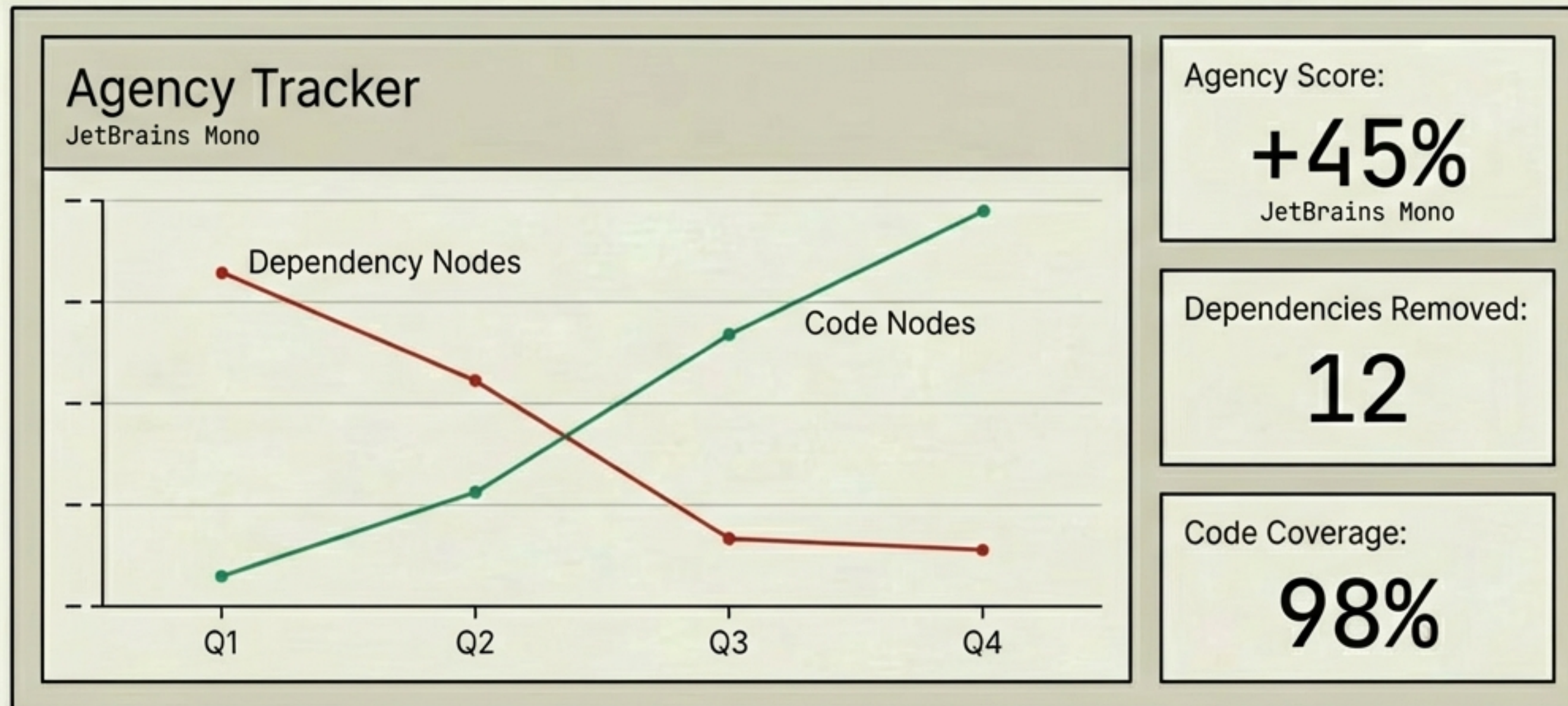
Fewer dependencies mean a smaller supply chain attack surface.
Every dependency removed is a security win.



Supply Chain Risk: Minimized.

Measuring the Shift with Issues FS

Tracking agency by mapping the dependency graph as issue types.



The Cost-Benefit Reality

Replacing a dependency isn't free. It requires ownership.

COSTS (Investment) JetBrains Mono	BENEFITS (ROI) JetBrains Mono
• LLM Token Costs • Developer Review Time • Increased Testing Effort	• Reduced Supply Chain Risk • Increased Agency & Control • No "Black Box" Surprises • Future-Proof Stability

The ROI is a robust, self-contained system that you actually understand.

Agency is a Choice

LLMs don't reduce your control over code. They increase it—if you have the courage to own it.

1. Map your tree.

2. Identify candidates for replacement.

3. Trade opacity for ownership.